



CUDA 실습

윤승현

동국대학교 컴퓨터·AI학부



배열의 덧셈

- CPU를 이용하여 두 배열의 합을 구하는 프로그램을 작성하고 시간을 측정
- GPU를 이용하여 두 배열의 합을 구하는 프로그램을 작성하고 시간을 측정
- 실행 결과와 시간 비교



배열의 덧셈(CPU)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void AddVecCPU(float *C, float *A, float *B, int size);

void main()
{
    int N = 1024;
    float *a = new float [N];
    float *b = new float [N];
    float *c = new float [N];
    for (int i = 0; i < N; ++i)
    {
        a[i] = rand() / (float)RAND_MAX;
        b[i] = -a[i];
        c[i] = 0.0;
    }

    clock_t st = clock();
    AddVecCPU(c, a, b, N);
    clock_t ed = clock();
    printf("Elasped time: %u ms\n", ed - st);

    for (int i = 0; i < N; ++i)
    {
        if (c[i] != 0.0)
            printf("Error\n");
    }

    delete [] a;
    delete [] b;
    delete [] c;
}
```

```
void AddVecCPU(float *C, float *A, float *B, int size)
{
    for (int i = 0; i < size; ++i)
        C[i] = A[i] + B[i];
}
```



배열의 덧셈(GPU)

```
#include <stdlib.h>
#include <stdio.h>
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

__global__ void AddVecGPU(float *c, float *a, float *b);

int main()
{
    int N = 256; // N의 최대값 = ?
    float *a = new float [N];
    float *b = new float [N];
    float *c = new float [N];
    for (int i = 0; i < N; ++i)
    {
        a[i] = rand() / (float)RAND_MAX;
        b[i] = -a[i];
        c[i] = 0.0;
    }

    // 사용할 GPU를 설정한다 (0번째 GPU)
    cudaError_t cudaStatus = cudaSetDevice(0);
    if (cudaStatus != cudaSuccess){
        printf("Error\n");
        return 1;
    }

    // GPU(Device)에 메모리를 할당한다.
    float *dev_a, *dev_b, *dev_c;
    cudaStatus = cudaMalloc((void **)&dev_a, N * sizeof(float));
    cudaStatus = cudaMalloc((void **)&dev_b, N * sizeof(float));
    cudaStatus = cudaMalloc((void **)&dev_c, N * sizeof(float));
```



배열의 덧셈(GPU)

```
// CPU 배열의 데이터를 GPU 메모리에 복사한다.
cudaStatus = cudaMemcpy(dev_a, a, N * sizeof(float), cudaMemcpyHostToDevice);
cudaStatus = cudaMemcpy(dev_b, b, N * sizeof(float), cudaMemcpyHostToDevice);
cudaStatus = cudaMemcpy(dev_c, c, N * sizeof(float), cudaMemcpyHostToDevice);

clock_t st = clock();
// N개의 쓰레드를 생성하여 AddVecGPU 커널을 실행한다.
AddVecGPU<<<1, N>>>>(dev_c, dev_a, dev_b);
// 모든 쓰레드가 동기화 될때까지 기다린다.
cudaStatus = cudaDeviceSynchronize();
clock_t ed = clock();
printf("Elapsed time: %u ms\n", ed - st);

// 결과를 GPU메모리에서 CPU메모리로 복사한다.
cudaStatus = cudaMemcpy(c, dev_c, N * sizeof(float), cudaMemcpyDeviceToHost);
for (int i = 0; i < N; ++i)
    if (c[i] != 0.0)
        printf("Error\n");

delete[] a;
delete[] b;
delete[] c;
cudaFree(dev_c);
cudaFree(dev_a);
cudaFree(dev_b);

return 0;
}
```

```
__global__ void AddVecGPU(float *c, float *a, float *b)
{
    int i = threadIdx.x;
    c[i] = a[i] + b[i];
    //printf("thread id = %d\n", i);
}
```

Q&A